

Uke 9

ORDER BY

- For å sortere radene i resultatet fra en SELECT-spørring
- Sorteringen er gjort i henhold til typens naturlige ordning
 - Tall: verdi
 - Tekst: alfabetisk
 - Tidspunkter: kronologisk osv.
- ORDER BY-klausulen kommer alltid etter WHERE-klausulen
- By default: asc (stigende)

Eks

```
SELECT f.title, f.prodyear
FROM film f
WHERE prodyear IS NOT NULL
ORDER BY prodyear desc, title asc
LIMIT 10;
```

title	prodyear
Hart's Lagoon	2011
2012: The War for Souls	2010
Alternadad	2010
Area 52	2010
Ark, The	2010
Arthur et la guerre des deux mondes	2010
Boob Job	2010
Captain Abdul's Pirate School	2010
Child 44	2010
China, I Love You	2010
(10 rows)	

Eks

```
SELECT f.title, f.prodyear
FROM film f
WHERE prodyear IS NOT NULL
ORDER BY prodyear desc, title desc
LIMIT 10;
```

title	prodyear
Hart's Lagoon	2011
Wolf of Wall Street, The	2010
Untitled Sam Raimi Project	2010
Untitled Penny Chenery/Secretariat Project	2010
Untitled Justice League of America Project	2010
Untitled Daphne Sheldrick Project	2010
Untitled Clive Barker/Edgar Allan Poe Project	2010
Underdog	2010
Undateable	2010
Torrente	2010
(10 rows)	

GROUP BY

- Tar en liste med kolonner, og grupperer dem i henhold til likhet på verdiene i disse kolonnene
- Vi kan så bruke aggregeringsfunksjoner på hver gruppe i SELECT-klausulen
- Vi kan da også ha de grupperende kolonnene sammen med aggregatet i SELECT-klausulen
- Kun de grupperte kolonnene gir mening å ha utenfor et aggregat i SELECT (Og de man SELECTer utenom aggregeringsfunksjoner må være med i group by)

Eks

```
SELECT f.title, f.prodyear  
FROM film f  
WHERE f.title = 'Così fan tutte';
```

title	prodyear
Così fan tutte	2005
Così fan tutte	1992
Così fan tutte	1992
Così fan tutte	1996
Così fan tutte	2000
Così fan tutte	1989
Così fan tutte	1988
Così fan tutte	1990
Così fan tutte	1994
Così fan tutte	1970
Così fan tutte	1984
Così fan tutte	1996
Così fan tutte	2000
Così fan tutte	2004

(14 rows)

```
SELECT f.title, f.prodyear, count(*)  
FROM film f  
WHERE f.title = 'Così fan tutte';  
GROUP BY f.title, f.prodyear;
```

title	prodyear	count
Così fan tutte	1970	1
Così fan tutte	1984	1
Così fan tutte	1988	1
Così fan tutte	1989	1
Così fan tutte	1990	1
Così fan tutte	1992	2
Così fan tutte	1994	1
Così fan tutte	1996	2
Così fan tutte	2000	2
Così fan tutte	2004	1
Così fan tutte	2005	1

(11 rows)

Nytt Eks:

Finn gjennomsnittsprisen for hver kategori

```
SELECT Category, avg(Price) AS Averageprice
FROM Products
GROUP BY Category
```

Resultat

Products				
ProductID (int)	Name (text)	Brand (text)	Price (float)	Category (text)
0	TV 50 inch	Sony	8999	Televisions
1	Laptop 2.5GHz	Lenovo	7499	Computers
2	Laptop 8GB RAM	HP	6999	Computers
3	Speaker 500	Bose	4999	Speakers
4	TV 48 inch	Panasonic	11999	Televisions
5	Laptop 1.5GHz	IPhone	5195	Computers

Finn gjennomsnittsprisen for hver kategori

```
SELECT Category, avg(Price) AS Averageprice
FROM Products
GROUP BY Category
```

Resultat: Velg ut kolonner og grupper iht. Categories

Price	Category
8999	Televisions
7499	Computers
6999	Computers
4999	Speakers
11999	Televisions
5195	Computers

Finn gjennomsnittsprisen for hver kategori

```
SELECT Category, avg(Price) AS Averageprice
FROM Products
GROUP BY Category
```

Resultat: Velg ut kolonner og grupper iht. Categories

Price	Category
8999	Televisions
11999	Televisions
7499	Computers
6999	Computers
5195	Computers
4999	Speakers

Finn gjennomsnittsprisen for hver kategori

```
SELECT Category, avg(Price) AS Averageprice  
FROM Products  
GROUP BY Category
```

Resultat: Regn ut aggregatet for hver gruppe og ferdigstill

avg(Price)	Category
10499	Televisions
6731	Computers
4999	Speakers

HAVING klausulen

- I enkelte tilfeller er vi kun interessert i grupper hvor et aggregat har en bestemt verdi
- F.eks. dersom man vil vite kategorinavn og antall produkter på de kategoriene som har flere enn 10 produkter
- Kommer alltid rett etter GROUP BY

- ◆ Denne klausulen heter **HAVING** og kommer rett etter **GROUP BY**, slik:

```
SELECT c.category_name, count(*) AS nr_products
FROM categories AS c
      INNER JOIN products AS p ON (c.category_id = p.category_id)
GROUP BY c.category_name
HAVING count(*) > 10;
```

- ◆ Merk: Kan ikke bruke navnene vi gir i **SELECT**
- ◆ **HAVING** blir altså evaluert på hver gruppe
- ◆ Fungerer altså som en slags **WHERE** for grupper

- ◆ Vi har nå sett mange nye klausuler
- ◆ Generelt ser våre SQL-spørringer nå slik ut:

```
    WITH <navngitte-spørringer>
SELECT <kolonner>
    FROM <tabeller>
    WHERE <uttrykk>
GROUP BY <kolonner>
    HAVING <uttrykk>
ORDER BY <kolonner> [DESC]
    LIMIT <N>
    OFFSET <M>
```

- ◆ I denne rekkefølgen (LIMIT og OFFSET kan bytte plass)
- ◆ Kan selvfølgelig droppe klausuler, men må ha GROUP BY for å ha HAVING